

Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	12 (Term 1)	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will develop their ability to inspect and process text in Python. They will identify character positions, use string length and select sections of text using slicing. Students will then apply string methods to normalise responses before comparing them. They will test empty input, case differences and surrounding spaces, explaining both the benefits and limitations of their solution.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, discussion of worked examples, the completed student tasks and the final checks in both lessons. Check correct indices and explain how the program avoids indexing an empty string. Test uppercase, lowercase, spaces and empty input; explain one limitation of the checker. Check that students can explain their reasoning and apply the skill without relying on the worked example.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	9 November 2026	By the end of this lesson, students will be able to use string length indexing and slicing safely.	**H:** Ask which position contains the first character in Python. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain zero-based indexing, len() and the excluded stop position in a slice. Explain that Python strings contain characters with zero-based positions. The length counts all characters, including spaces, while a slice selects a sequence up to but not including its stop position. **WD:** Predict the results of indexing and slicing a supplied word. Use the word COMPUTER to label positions 0 to 7. Predict word[0], word[1:4] and len(word), then run them. Show why requesting the first character of an empty string is unsafe.	**Support:** Provide an indexed character strip and partially completed expression table. Model counting positions from zero and shade the characters in a slice. Supply the empty-input guard before independent practice. **Extension:** Students compare a slice extending beyond the string with an out-of-range single index. They describe the observed difference and propose a test that demonstrates it clearly.

			YD: Write a program that displays a word length, its first character and a selected slice; check for empty input first. Students produce a small table of expressions, predictions and actual results. Their program checks for an empty response before accessing a character, and they explain which positions are included in their chosen slice. **C:** Check correct indices and explain how the program avoids indexing an empty string. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
2	13 Novembe r 2026	By the end of this lesson, students will be able to apply string operations to normalise and inspect text.	**H:** Compare " YES " with "yes" in a decision. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain strip(), lower() and membership tests without changing the original string in place. Explain that normalisation makes comparable responses consistent and that string methods return new strings. Discuss when removing surrounding spaces and changing case are useful and when the original text should be retained. **WD:** Build a fictional club-code checker using a normalised input. Store the original response " YES " and a normalised version using strip().lower(). Compare the two and use a condition to recognise an accepted response.	**Support:** Provide a before-and-after table for strip() and lower(). Separate the method calls into two assignments before combining them. Give a response list so students focus on processing rather than inventing rules. **Extension:** Students test a response containing internal spaces and explain why strip() does not remove them. They describe a limitation without claiming the checker can interpret any natural-language answer.

			Demonstrate that an empty normalised response needs separate feedback. **YD:** Create a response checker that accepts mixed-case input with surrounding spaces and reports empty input. Students define accepted responses before coding and test mixed case, surrounding spaces, an incorrect response and empty input. They keep expected and actual results and explain which requirement each string operation supports. **C:** Test uppercase, lowercase, spaces and empty input; explain one limitation of the checker. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Pearson Edexcel International GCSE Computer Science Student Book: Chapter 7
- Teacher computer and classroom display
- Classroom computers with Python 3 editor and individual save folders
- Prepared algorithm, tracing or Python practice tasks aligned with the lesson objectives
- Whiteboard and markers
- Student notebooks and pencils
- Success criteria, test tables and teacher observation checklist



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?